Applied Soft Computing 138 (2023) 110175

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

FedAGCN: A traffic flow prediction framework based on federated learning and Asynchronous Graph Convolutional Network

Tao Qi [a], Lingqiang Chen [a], Guanghui Li [a,*], Yijing Li [a], Chenshu Wang [b]

[a] School of Artificial Intelligence and Computer Science, Jiangnan University, Wuxi, Jiangsu 214122, China
[b] School of Computing Science and Communication Engineering, Jiangsu University, Zhenjiang, Jiangsu, 212000, China

ARTICLE INFO

Article history:
Received 19 April 2022
Received in revised form 26 January 2023
Accepted 3 March 2023
Available online 8 March 2023

Keywords:
Traffic flow prediction
Graph convolutional network
Asynchronous spatial–temporal correlation
Federated learning

ABSTRACT

Accurate and real-time traffic flow prediction is an essential component of the Intelligent Transportation System (ITS). Balancing the prediction accuracy and time cost of prediction models is a challenging topic. This paper proposes a deep learning framework (FedAGCN) based on federated learning and asynchronous graph convolutional networks to predict traffic flow accurately in real time. FedAGCN applies asynchronous spatial–temporal graph convolution to model the spatial–temporal dependence in traffic data. In order to reduce the time cost of the deep learning model, we propose a graph federated learning strategy GraphFed to train the model. Experiments were conducted on two public traffic datasets, and the results showed that FedAGCN effectively reduced the training and inference time of the model while maintaining considerable prediction accuracy.

1. Introduction

Modern cities are gradually transforming into smart cities. The acceleration of urbanization and the growth of per capita vehicle ownership put tremendous pressure on the city's comprehensive management [1]. Traffic management systems can alleviate traffic congestion, exhaust pollution, and traffic accidents. With the rapid development of computer systems in recent years, intelligent transportation systems (ITS) are playing an increasingly important role in urban traffic management and smart city construction [2]. Traffic flow prediction is the basis of intelligent transportation systems, especially short-term traffic flow prediction. Accurate and real-time short-term traffic flow prediction is essential for many applications. For example, on the Internet of Vehicles, accurate traffic flow prediction allows drivers to understand the surrounding road conditions and effectively improve the driving experience [3]; In the navigation system, through accurate and real-time travel time prediction, drivers can obtain more intelligent navigation decisions to avoid entering potentially busy roads [4].

With the continuous growth of available datasets related to traffic and the vigorous development of deep learning, researchers began to apply deep learning on traffic prediction tasks. Traffic data show an apparent spatial–temporal correlation [5]. Hence spatial–temporal correlation modeling in traffic data is the leading research direction for researchers. Recent studies mostly use the graph neural network (GNN) to model the spatial relationship of traffic data and use the recurrent neural network (RNN) or convolutional neural network (CNN) to model the temporal relationship of traffic data [6–8]. These methods effectively construct the spatial and temporal relationship in traffic networks and have achieved good results in various spatial–temporal prediction issues. However, deep learning models require enormous computational resources, and their deployment cost and training time are high. In practical application, the training time, inference time, and deployment cost of the model have a significant impact on the actual effect of the model.

In the task of short-term traffic flow prediction, the time cost of the model is mainly composed of the following three parts: (1) data transmission cost; (2) model training time; (3) model inference time. The cost of data transmission mainly refers to the time and bandwidth cost of transmitting data from various transportation nodes distributed in the transportation network to the cloud server. Due to the continuous growth of the transportation network and the increasing traffic data, the cost of this part is becoming higher and higher. In addition, data transmission costs are easily affected by external factors such as weather and network fluctuations. The training time and inference time of the model are also affected by the transmission cost. The existing centralized model needs to obtain traffic data from each traffic detector, which requires a large amount of bandwidth to transmit the data to the central server, resulting in an inevitable time delay.

* Corresponding author.
E-mail addresses: taoqi@stu.jiangnan.edu.cn (T. Qi), lqchen@stu.jiangnan.edu.cn (L. Chen), ghli@jiangnan.edu.cn (G. Li), 6191910035@stu.jiangnan.edu.cn (Y. Li), 2211908036@stmail.ujs.edu.cn (C. Wang).

https://doi.org/10.1016/j.asoc.2023.110175
1568-4946/

Federated learning can help solve the above issues. In [9], McMahan et al. first proposed the concept of federated learning. Specifically, federated learning adopts a distributed training strategy, which solves the transmission delay caused by excessive data. At the same time, dividing the centralized model into multiple sub-models for parallel training can effectively reduce the training time. However, there are the following challenges in applying the original federated learning model to the traffic prediction task: (1) in the original federated learning model, each client only processes its local data, so it fails to extract the spatial association in the traffic data by using the topology of the traffic network; (2) After each round of training, federated learning needs to upload the parameters of each traffic node participating in this round of training to the cloud to obtain the cloud model. However, in the traffic prediction task, the spatial correlation of traffic data has an apparent locality; in other words, traffic data from different areas own different statistical characteristics. Graph convolution network is usually used to model the correlation of traffic data. Aggregating the parameters of the graph convolution network in each sub-model will weaken the local spatial relationship, resulting in a degradation in the prediction performance of the model.

In this article, we propose the FedAGCN model to solve the above problems. FedAGCN uses the improved federated learning strategy to train the predictive model. First, FedAGCN uses a clustering algorithm to divide the entire transportation network into K sub-graphs. Each sub-graph is regarded as a trainable client to train a sub-model, effectively using the topological structure of the transportation network. In addition, we propose a graph federated learning strategy GraphFed to solve the parameter update problem mentioned above. Our main contributions are as follows:

(1) We design a sub-graph partition strategy based on a clustering algorithm, which effectively constructs the spatial correlation in the traffic data by treating each sub-graph as a trainable client and using the topological structure of the transportation network.
(2) We propose a graph federated learning strategy GraphFed that decomposes the parameter update process in federated learning into local and global updates. The GraphFed strategy avoids the loss of local spatial correlation during parameter updates and ensures the model's predictive performance.
(3) We conduct detailed experiments of the proposed model on two real-world traffic datasets: METR-LA and PEMS08. Compared with the existing deep learning model, FedAGCN can significantly reduce the model's training time and inference time while ensuring considerable prediction accuracy.

The remainder of this paper is organized as follows. Section 2 covers the literature on traffic prediction. Section 3 introduces the preliminary knowledge in this paper, and then we detail the proposed method in Section 4. In Section 5, we evaluate the predictive performance of the FedAGCN. Finally, we conclude the paper in Section 6.

2. Related work

2.1. Traffic flow prediction

In recent years, artificial intelligence algorithms based on big data have gradually developed. Some researchers have used linear regression models in the field of traffic prediction, such as the historical average model (HA) [10] and the autoregressive integrated moving average model (ARIMA) [11]. However, these models assume that the target data is stable, so they cannot effectively deal with non-linear traffic flow data. Therefore, non-linear models have become a hot spot for studying traffic flow data, including support vector machines (SVM) [12], autoencoder models [13], and neural network models. Among them, the convolutional neural network (CNN) has apparent advantages in processing nonlinearity and extracting dynamic data features. In recent years, it has been widely used in non-linear traffic flow forecasting, and its forecasting performance is better than traditional non-linear forecasting models. However, CNN-based methods have apparent limitations when dealing with non-Euclidean structured data.

CNN regard the spatial correlation of traffic data as a simple Euclidean structure, ignoring complex spatial correlations. The emergence of graph convolutional networks (GCN) can help overcome this limitation. GCN uses the graph fourier transform to define the convolution operation in the frequency domain to realize the convolution operation of non-European structure data. Zhang et al. [14] combined GCN and a novel SortPooling layer to perform graph classification tasks. Johnson et al. [15] used graph convolution to process the input graphs and proposed a method for generating images from scene graphs. In order to extract the temporal characteristics of traffic data, researchers combined graph convolutional neural networks with recurrent neural networks (RNN) or convolutional networks to achieve better prediction performance. Yu et al. [16] proposed STGCN module, which used graph convolutional neural networks to extract spatial features while using CNN to extract temporal features. Experimental results show that the prediction performance of STGCN is better than traditional methods. Zhao et al. [17] proposed a Time Graph Convolutional Network (T-GCN) model, representing the traffic network as a graph structure, and combined GCN and GRU to model the spatial–temporal dependence of traffic data. Li et al. [18] first modeled traffic flow as a diffusion process on a directed graph and proposed a Diffusion Convolutional Recurrent Neural Network (DCRNN) model. Based on DCRNN, Wu et al. [19] proposed Graph Wavenet, which extracted spatial–temporal features through diffusion convolutional networks and dilated causal convolutional networks. Different from DCRNN, Graph Wavenet achieved lower training time. Cui et al. [20] proposed a Graph Wavelet Gated Recurrent Neural Network (GWGR). Graph wavelet can make the model focus on the key vertices in the graph and extract localized features discriminately.

In addition, the attention mechanism also plays an important role in traffic flow prediction. The attention mechanism was originally proposed in the field of natural language processing and has been widely used in many research fields. The traffic state of a traffic node is affected to different degrees by other nodes, and this influence often changes with the flow of time and different spatial locations, and is highly dynamic. Pan et al. [21] proposed the ST-MetaNet model based on deep meta-learning, which adopts an encoder–decoder architecture. The encoder and decoder have the same network structure, where a recurrent neural network is used to encode the input sequence and a meta-graph attention network is used to capture different spatial correlations. Zheng et al. [22] proposed a Graph multi-attention network (GMAN). GMAN also adopts an encoder–decoder architecture, where the encoder is used to encode the input sequence and the decoder is used to predict the output sequence. The difference is that GMAN simultaneously uses an attention mechanism to model the spatiotemporal connections of traffic data. Experimental results show that GMAN has high prediction accuracy, especially in long-term prediction tasks. However, due to the high computational complexity of the attention mechanism, the training time and inference time of GMAN are higher than most GCN-based traffic prediction models.

The methods mentioned above have achieved great success in traffic prediction tasks. However, few researchers pay attention to

the deployment cost and time spent on the model. In this study, we divide the traffic network into K sub-graphs and propose FedAGCN model, which minimizes the model's deployment cost and time consumption while ensuring the prediction accuracy of the model.

2.2. Federated learning

Deep learning models are mostly deployed in the cloud. It is necessary to upload the relevant information to the cloud for a further calculation. This process still requires a large amount of network traffic and consumes communication costs. Google [9] first proposed the concept of federated learning. Federated learning is a distributed training network structure, but unlike regular distributed training, the computing nodes of federated learning have absolute control over the data. With the rapid development of edge computing in recent years, federated learning is one of the forms of cloud–edge collaborative computing scenarios, and its related research is also increasing. Federated learning methods have been deployed in service providers [23,24] and have played an important role in many applications. Ammad [25] and others combined collaborative filtering and federated learning for the first time and explored a personalized recommendation system based on implicit user feedback. In addition, federated learning also plays an important role in fields such as natural language processing-related predictions [26,27], smart medical [28,29], smart home [30], and traffic prediction. Liu et al. [31] proposed the FedGRU model, which uses federated learning and GRU to solve the traffic prediction problem.

The above models rarely consider the spatial correlations between edge nodes, and the prediction accuracy of the model also needs to be improved. We apply federated learning to traffic prediction tasks and propose a GraphFed strategy to utilize the spatial correlations in traffic data.

3. Preliminaries

3.1. Traffic flow forecast

The traffic network is regarded as a spatial graph, which can be expressed as $G = (\mathcal{V}, \mathcal{E}, \mathbf{A})$, where $\mathcal{V}$ is a set of road nodes, $\mathcal{E}$ is a set of edges, $\mathbf{A} \in \mathbb{R}^{N\times N}$ is adjacency matrix of the graph, which represents the connection between nodes. If $v_i, v_j \in \mathcal{V}$ and $(v_i, v_j) \in \mathcal{E}$, then the matrix element a_{ij} is 1 otherwise it is 0. At each time step t, the spatial graph G has a feature matrix $\mathbf{X}^{(t)} \in \mathbb{R}^{N\times D}$. Given the feature matrix of the graph G and its S historical time steps, the task is to predict the feature matrix of the graph G in the next T time steps. The mapping relationship can be expressed as:

$$\left[\mathbf{X}^{(t-S+1):t}, G\right] \xrightarrow{f} \mathbf{X}^{(t+1):(t+T)} \tag{1}$$

where $\mathbf{X}^{(t-S+1):t} \in \mathbb{R}^{N\times D\times S}$ is the input of the model, $\mathbf{X}^{(t+1):(t+T)} \in \mathbb{R}^{N\times D\times T}$ is the prediction output of the model, and f is the model we learned.

3.2. Graph convolution operation

Graph convolution networks can be divided into spectral domain graph convolution and spatial domain graph convolution [32]. Spectral domain graph convolution defines graph convolution by introducing a filter from the perspective of graph signal processing, which interprets the graph convolution operation as removing noise from the graph signal; The convolution operation of the spatial domain is defined by way of information propagation. The model in this paper uses graph convolution operation based on spatial domain, and its expression is:

$$h^{(l)} = \mathbf{A}h^{(l-1)}W + b \tag{2}$$

where $h^{(l)}$ represents the output of the lth graph convolutional layer, $h^{(l-1)} \in \mathbb{R}^{N\times C}$ is the input of the lth convolutional layer. When $l = 1$, $h^{(0)}$ is $\mathbf{X}$, that is, the feature matrix of traffic data. $W \in \mathbb{R}^{C\times C'}$ and $b \in \mathbb{R}^{C'}$ are trainable parameters. $\mathbf{A}$ is the adjacency matrix of the spatial graph of the traffic road network. For any traffic node $v_i \in \mathcal{V}$, a graph convolution operation is equivalent to aggregating the information of its first-order neighbors and itself. In order to expand the receptive field of the convolution operation, all graph convolution layers can be stacked to aggregate the information of the l neighbors of the node v_i.

3.3. Federated learning

Federated learning [9] is a distributed machine learning method designed to avoid privacy disclosure. In federated learning, different organizations can participate in the training of cloud model without uploading their unique data to the cloud. Using federated learning, we can effectively reduce the impact of network fluctuation and avoid massive data transmission during the model training process. Therefore, the training time are saved.

In the traditional federated learning method, it is generally assumed that there is a set of computing devices $\mathbf{E}$ of memory size K. Each device stores its local data $\mathbf{L}_k$ with a size of D_k. Hence the total amount of data is $D = \sum_{k=1}^{K} D_k$. For a certain computing device k, use $\{x_i, y_i\}_{i=1}^{D_k}$ to represent the input and label of a sub-model. Then the loss of the sub-model on the computing device k can be expressed as $f_i(w, x_i, y_i)$. The loss function of the sub-model on this device can be expressed as:

$$J_k(\omega) := \frac{1}{D_k}\sum_{i=1}^{D_k} f_i(\omega) + \lambda h(\omega) \tag{3}$$

Among them, $\omega \in \mathbb{R}^d$ is the parameter of the sub-model, and $\lambda h(\omega)$ is the regularization term. The cloud updates the cloud model by aggregating the sub-model parameters on each device. The loss function of the cloud model is:

$$\arg\min_{\omega\in\mathbb{R}^d} J(\omega) := \sum_{k=1}^{K} \frac{\sum_{i=1}^{D_k} f_i(\omega) + \lambda h(\omega)}{D} \tag{4}$$

4. Methodology

4.1. Traffic network partition method based on clustering

The spatial correlation in the traffic data shows apparent locality. For example, in a certain area, there is a strong interaction between traffic nodes due to the influence of road network structure. GCN can model the spatial correlation of traffic network. However, when the traffic network covers a large area, GCN will need a huge adjacency matrix to describe the topology of the traffic graph. This will cause the model to take up more computing resources. In addition, aggregating a wide range of node information also makes the model more prone to over-smoothing. Therefore, we try to train a predictive model using a federated learning strategy. In the previous literature, when applying federated learning methods to deal with traffic prediction problems, each traffic node is often regarded as a client that can participate in model training [31]. This approach ignores the spatial structure of the traffic network and the spatial correlation

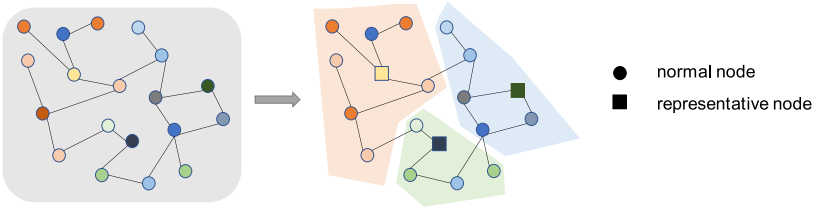


Fig. 1. Sub-graph partition.

of traffic data. Compared with the existing centralized model, its prediction performance is not satisfactory.

To solve the above problems, we use a graph clustering algorithm to divide the spatial graph of the traffic road network into multiple sub-graphs. The goal of graph clustering algorithms, such as Metis [33] or Graclu [34], is to divide the vertices in the graph into multiple clusters so that the number of edges in a cluster is much greater than the number of edges between clusters. Compared with the simple division based on the location or distance of traffic nodes, data-driven division methods can better retain the spatial–temporal correlations in the sub-graphs after division. As a result, the prediction accuracy of the model can be retained to the greatest extent.

As shown in Fig. 1, given a traffic network $G = (\mathbf{V}, \mathbf{E}, \mathbf{A})$, we can divide the original graph structure into n sub-graphs, i.e. $G = \{G_0, G_1, \ldots, G_n\}$, where $G_i = (\mathbf{V_i}, \mathbf{E_i}, \mathbf{A_i}), i \in [0, n]$. A sub-graph contains several traffic nodes (circle), any one of them $v_c \in \mathbf{V_i}$ is deployed as the representative node (square) of the sub-graph. The set of all representative nodes is represented as $\mathbf{V_c}$. The representative node has the graph topology information of the sub-graph where it is located and the data collected by other ordinary nodes, and train a sub-model based on this data.

4.2. GraphFed: Graph federation learning strategy

The centralized deep learning model needs to upload data from traffic nodes distributed throughout the transportation network to the cloud, which will cause a considerable communication load. Furthermore, the time required for data transmission between the cloud and traffic nodes will seriously affect the training time of the model. However, the traditional federated learning algorithms cannot make use of the spatial structure of the traffic network, and its prediction accuracy is poor compared with the existing deep learning model. To this end, we proposes a new federated learning strategy (GraphFed). The GraphFed algorithm includes the following steps:

Step 1. Divide the traffic road network into multiple sub-graphs using the traffic network division method mentioned in the previous subsection. It can be expressed as $G = \{G_0, G_1, \ldots, G_n\}$. In each sub-graph, we choose a node v_c as the representative node of the sub-graph. Then a sub-model is created on each representative node. Note that, unlike traditional federated learning strategies, GraphFed divides the parameters of the sub-model into two categories: local parameters p_l and global parameters p_g. Local parameters describe the spatial dependence of traffic data, for example, W and b in Eq. (2), and all other parameters are global parameters. Since each representative node is responsible for different sub-graphs, the local parameters p_l are different in each representative node.

Step 2. The cloud selects some nodes $\mathcal{V}_o$ from the representative node-set $\mathbf{V_c}$, we called volunteer nodes, to participate in this round of training and distributes the global parameters of the cloud model p_g to the volunteer nodes. The cloud model does not contain local parameters, and the local parameters p_l of each sub-model are initialized by themselves at the beginning of training. Specifically, each representative node randomly initializes its local parameters at the model initialization, and uses previously learned local parameters for the next training process.

Step 3. The sub-model on each volunteer node is trained using the stochastic gradient descent method and the model parameters p_l^t and p_g^t are updated. The training process only uses the local data of the sub-graph. It can be expressed as:

$$p_l^{t+1}, p_g^{t+1} \leftarrow \text{LocalUpdate}\left(v_o, p_l^t, p_g^t\right) \tag{5}$$

this process will be executed E rounds in parallel on each volunteer node.

Step 4. Each volunteer node saves its own trained local parameter p_l^{t+E}, and then uploads the global parameter p_g^{t+E} to the cloud. The cloud aggregates the global parameters of all volunteer nodes and updates them to the global parameters of the cloud model. In this step, there are a wealth of parameter aggregation mechanisms available for use. For the sake of simplicity, we uses the average aggregation method.

The GraphFed algorithm is the core mechanism of the FedAGCN model, which can effectively reduce the calculation time of the model and the communication overhead while ensuring high prediction accuracy. The algorithm is iterative and will continue to repeat steps 2–4 until the cloud model converged.

4.3. Improved asynchronous spatial–temporal graph convolutional network based on federated learning

From the perspective of information dissemination, the temporal dependence of the same traffic node at different times can be regarded as the information dissemination process on the graph. The spatial–temporal dependence between nodes can also be represented by the edges on multiple spatial graphs. The ADGCN [35] model can intuitively capture the spatial–temporal correlations in the transportation network and accurately predict the state of the transportation network. Fig. 2 shows the model framework of ADGCN. The model constructs an asynchronous spatial–temporal correlation matrix ASTCM from the original data and an asynchronous spatial–temporal graph convolution layer based on this matrix. First, the input layer transforms the original data into the high-dimensional feature space. Then the spatial–temporal features are extracted through the multi-layer asynchronous spatial–temporal graph convolution layer. Finally, the output layer generates the predicted output of S time steps. However, the network structure of ADGCN is more complex and requires more parameters. To make the model more lightweight, we improve the graph convolution operation of ADGCN, call the new model AGCN, and use it as a sub-model of FedAGCN.

Given a volunteer node $\mathbf{v_o} \in \mathbf{V_c}$, and its sub-graph is G_o. We arbitrarily select m ($m \leq S$) continuous spatial graph $\mathbb{G}_o^{(t_0:t_m)} = \left(G_o^{(t_0)}, G_o^{(t_1)}, \ldots, G_o^{(t_m)}\right)$ in historical observation data of S time

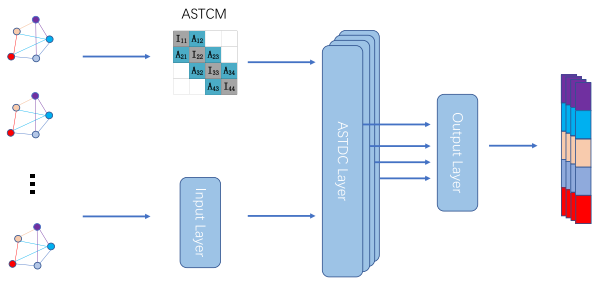


Fig. 2. ADGCN model framework.

steps, then the asynchronous spatial–temporal correlation graph can be expressed as $G_o^{(H)} = \left(\mathcal{V}_o^{(H)}, \mathcal{E}_o^{(H)}, A_o^{(H)}\right)$, where $\mathcal{V}_o^{(H)}$ and $\mathcal{E}_o^{(H)}$ respectively represent the vertex set and edge set in the asynchronous spatial–temporal correlation graph. $A_o^{(H)}$ is the Asynchronous Spatial-Temporal Correlation Matrix, defined as follows:

$$A_o^{(H)} = \begin{pmatrix} I & A & 0 & \cdots & 0 & 0 & 0 \\ A & I & A & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & I & A & 0 \\ 0 & 0 & 0 & \cdots & A & I & A \\ 0 & 0 & 0 & \cdots & 0 & A & I \end{pmatrix} \in \mathbb{R}^{mN \times mN} \tag{6}$$

where I is the identity matrix and A is the adjacency matrix of the spatial graph.

In the original ADGCN model, combining multiple spatial graphs into asynchronous spatial–temporal graphs will lead to a rapid expansion of the network scale and a sharp increase in the number of parameters required by the model, making the model over-fitting and difficult to train. GraphFed algorithm solves this problem. Suppose there are N nodes in the traffic road network, and m ($m \leq S$) spatial graphs are used to construct an asynchronous spatial–temporal graph. Then the parameter amount required for the convolution operation of ADGCN is $mN \times mN$. GraphFed algorithm divides the traffic road network into n sub-graphs. Assuming that the number of nodes in each sub-graph is the same, that is, $\frac{N}{n}$, then the amount of parameters required for the graph convolution operation of each sub-model is $m\frac{N}{n} \times m\frac{N}{n}$. In general, the amount of parameters required for the graph convolution operation of all sub-models is:

$$P_{all} = \left(m\frac{N}{n} \times m\frac{N}{n}\right) \times n \tag{7}$$

Usually, due to the limitation of spatial distance, we need to divide as much sub-graphs as possible. It means that the value of n is generally larger. Meanwhile, m is much less than n. It can be seen from Eq. (7) that the parameter amount of the sub-model on each sub-graph does not increase sharply with the increase of m. We can take $m = S$ and directly use all the spatial graphs to construct an asynchronous spatial–temporal graph. Only the graph convolution operation is used to model the asynchronous spatial–temporal correlation in the traffic data.

To accurately learn the strength of the asynchronous spatial–temporal correlation in traffic data, we added an adaptive asynchronous spatial–temporal correlation weight matrix $A_o^{(adp)}$ to participate in the training. In order to further reduce the amount of parameters, instead of directly initializing a parameter matrix with the same size as $A_o^{(H)}$, a tensor D_o of length $m\frac{N}{n}$ is initialized, then $A_o^{(adp)}$ can be defined as:

$$A_o^{(adp)} = softmax(RELU(D_o D_o^T)) \tag{8}$$

The graph convolution operation on the graph G_o is:

$$h_o^{(l)} = \mathbf{A}_o h_o^{(l-1)} W_o + b_o \tag{9}$$

$$\mathbf{A}_o = A_o^{(adp)} \odot A_o^{(H)} \tag{10}$$

Among them, $\odot$ means element-wise multiplication.

To expand the receptive field of graph convolution operation, multi-layer asynchronous spatial–temporal graph convolution is stacked together to form an asynchronous spatial–temporal graph convolution block. In addition, in order to solve the smoothing problem, a gated fusion layer is designed to fuse the results of each layer of convolution operation:

$$h_o^f = g\left(\mathbf{W_o^1}[h_o^1, h_o^2, \ldots, h_o^l] + \mathbf{b}_o\right) \tag{11}$$

$$h_o^g = \sigma\left(\mathbf{W_o^2}[h_o^1, h_o^2, \ldots, h_o^l] + \mathbf{c}_o\right) \tag{12}$$

$$h_o^M = h_o^f \odot h_o^g \tag{13}$$

Among them, $\mathbf{W_o^1}$, $\mathbf{W_o^2}$, $\mathbf{b}_o$ and $\mathbf{c}_o$ are learnable parameters, h_o^l is the output feature of each convolutional layer, $[\cdot]$ represents the connection operation, $g(\cdot)$ is the activation function, such as relu or tanh, $\sigma(\cdot)$ is the sigmoid function, which determines the ratio of the information input to the next layer, and $\odot$ is element-wise multiplication.

We stack multiple layers of asynchronous spatial–temporal graph convolution blocks, and the output of each layer is used as the input of the next layer. The output of the last layer is represented by H_o^s. We use a two-layer fully connected network as the output layer to generate predictions for T time steps:

$$\hat{y}_o = \text{ReLU}\left(H_o^s W_o^1 + b_o^1\right) \cdot W_o^2 + b_o^2 \tag{14}$$

Among them, $\hat{y}_o \in \mathbf{R}^{N \times T}$ is the predicted output of the model. W_o^1, W_o^2, b_o^1, b_o^2 are trainable parameters.

From Eq. (3), the loss function of the sub-model for representative node k is:

$$J_k(p_l) := \frac{1}{D_k} \sum_{i \in D_k} f_i(p_l) + \lambda h(p_l) \tag{15}$$

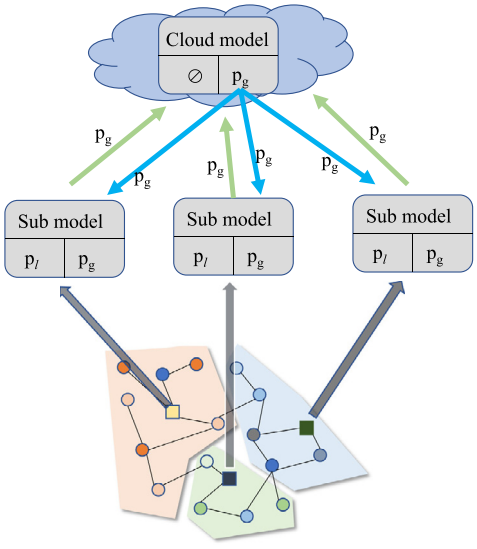


Fig. 3. FedAGCN model framework.

According to Eq. (4), the loss function of the cloud model is:

$$\arg\min_{p_l \in R^d} J(p_l) := \sum_{k=1}^{K} \frac{\sum_{i=1}^{D_k} f_i(p_g) + \lambda h(p_l)}{D} \tag{16}$$

The model architecture of FedAGCN is shown in Fig. 3. The cloud model is only used to aggregate the global parameters of each sub-model. After the model training is completed, each representative node performs the traffic flow prediction task on its own sub-graph.

5. Experiments

We conducted experiments on two real-world datasets and verified the effectiveness of the FedAGCN model by comparing it with several latest traffic flow prediction models using deep learning.

5.1. Dataset description

We validated our model on two public traffic datasets, PEMS08 and METR-LA.

- METR-LA [18]: METR-LA records the traffic speed information of 207 loop detectors on the highways of Los Angeles County, whose frequently referenced period is Mar 1st, 2012, to Jun 30th, 2012.
- PEMS08 [36]: The Caltrans Performance Measurement System (PEMS) deploys more than 39 000 traffic detectors on highways in major metropolitan areas of California, and obtains 30-s loop detector data in real time. PEMS08 contains 170 detectors on eight roads in the San Bernardino area, and its frequently referenced period is July to August 2016.

Our data preprocessing method is consistent with Graph Wavenet [19]. First, we aggregate the average vehicle speed readings with a time window of 5 min and then apply Z-Score to normalize them:

$$Z = \frac{X - \mu_X}{\sigma_X} \tag{17}$$

where X is the sample to be processed, μ_X represents the average value of the sample, and σ_X is the standard deviation of the sample. Table 1 shows the details of the processed datasets. In addition, we divide the data into the training set, validation set, and test set according to the ratio of 7 : 2 : 1. In order to construct the adjacency matrix of the traffic road network, the distance of the paired road network between the detectors is calculated, and the detectors that are too close to each other are removed. Subsequently, the threshold Gaussian kernel [37] combined with Eq. (6) was used to construct the adjacency matrix, and the threshold Gaussian kernel calculation formula is as follows:

$$W_{ij} = \begin{cases} \exp\left(-\frac{d_{ij}^2}{\sigma^2}\right) & d_{ij} \le \epsilon \\ 0 & d_{ij} > \epsilon \end{cases} \tag{18}$$

where d_{ij} represents the distance between detector i and detector j, σ is the standard deviation of the distance, and ϵ is the threshold. The larger the W_{ij}, the more relevant the vertex i and the vertex j. The values of σ and ϵ are the same as the settings in Graph Wavenet [19].

Table 1
Summary statistics of datasets.

Data	Nodes	Time range
METR-LA	207	3/1/2012–6/30/2012
PEMS08	170	7/1/2016–8/31/2016

5.2. Baselines

We compare FedAGCN with the following models.

- T-GCN [17]: Temporal Graph Convolutional Network. This model uses a gated recurrent unit to process temporal information and a graph convolutional network to process spatial information.
- DCRNN [18]: Diffusion Convolutional Recurrent Neural Network. It models the traffic flow as a diffusion process on a directed graph.
- Graph Wavenet [19]: A powerful model combines diffusion convolution and adaptive adjacency matrix to extract spatial dependence and uses dilated causal convolution to deal with temporal dependence.
- GMAN [22]: Graph Multi-Attention Network, this model adapts an encoder–decoder architecture and uses an attention mechanism to model the impact of spatial–temporal factors on traffic conditions.

5.3. Evaluation metrics

Our experiment uses Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), training time, and inference time to evaluate the prediction performance of the FedAGCN model:

(1) Mean Absolute Error

$$MAE = \frac{1}{n}\sum_{i=1}^{n} \left|y_i - \hat{y}_i\right| \tag{19}$$

(2) Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n} \left(y_i - \hat{y}_i\right)^2} \tag{20}$$

(3) Mean Absolute Percentage Error

$$MAPE = \frac{100\%}{n}\sum_{i=1}^{n} \left|\frac{\hat{y}_i - y_i}{y_i}\right| \tag{21}$$

(4) Training time

$$T_{train} = \frac{\sum_{e=1}^{E} T_{train}^{e}}{E} \tag{22}$$

(5) Inference time

$$T_{inf} = \frac{\sum_{e=1}^{E} T_{inf}^{e}}{E} \tag{23}$$

where y_i and $\hat{y}_i$ are the real and the predicted values of traffic data at ith time step, respectively. n is the number of time steps. E is the number of rounds required for model training, and T_{train}^{e} is the time spent in the eth round of training. In addition, the inference time is calculated using the average inference time of the model on the test set, T_{inf}^{e} is the time spent in the eth round of inference.

5.4. Model parameters selection

There are two essential hyperparameters of the FedAGCN model: the number of connected spatial graphs in the asynchronous spatial–temporal graph and the number of sub-graphs. We conducted detailed experiments to determine the optimal values of these two parameters.

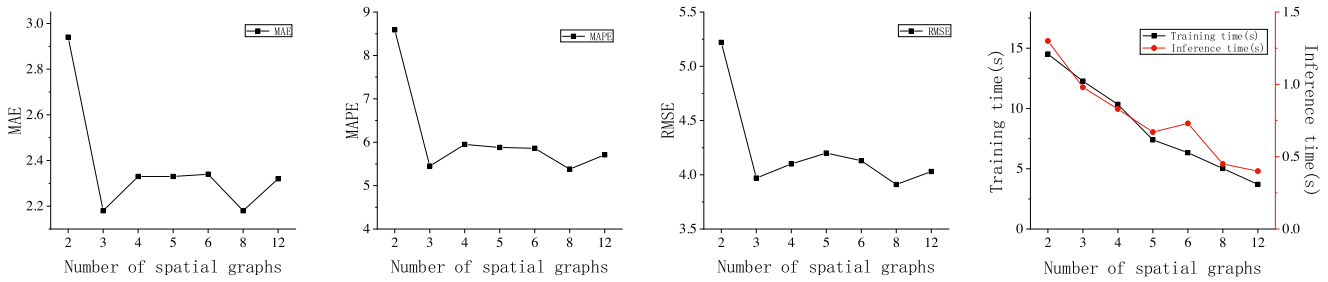


Fig. 4. Comparison of predicted performance over different number of connected spatial graphs in the validation set based on METR-LA dataset.

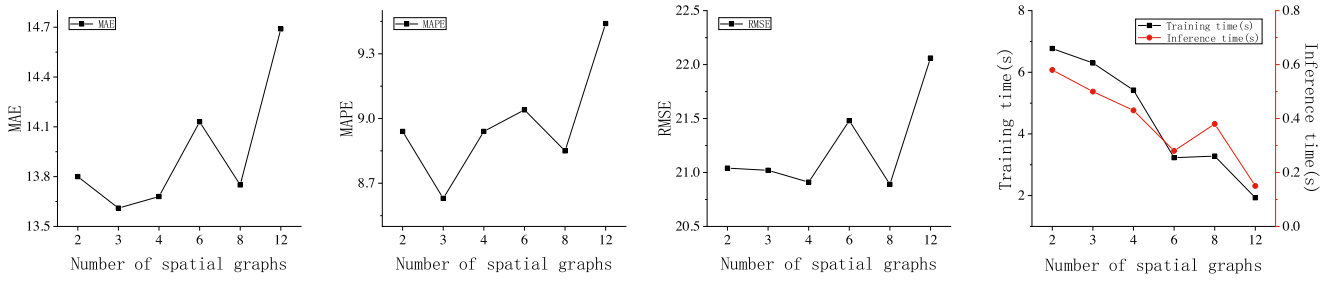


Fig. 5. Comparison of predicted performance over different number of connected spatial graphs in the validation set based on PeMS08 dataset.

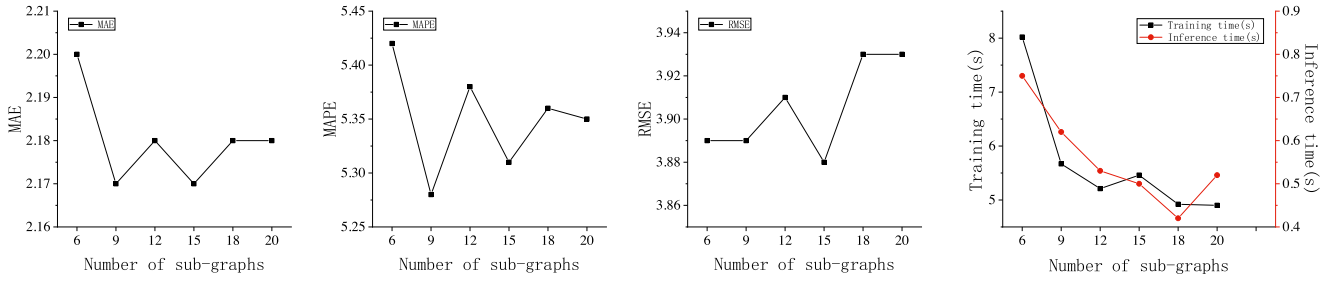


Fig. 6. Comparison of predicted performance over different number of sub-graphs [6, 9, 12, 15, 18, 20] for METR-LA.

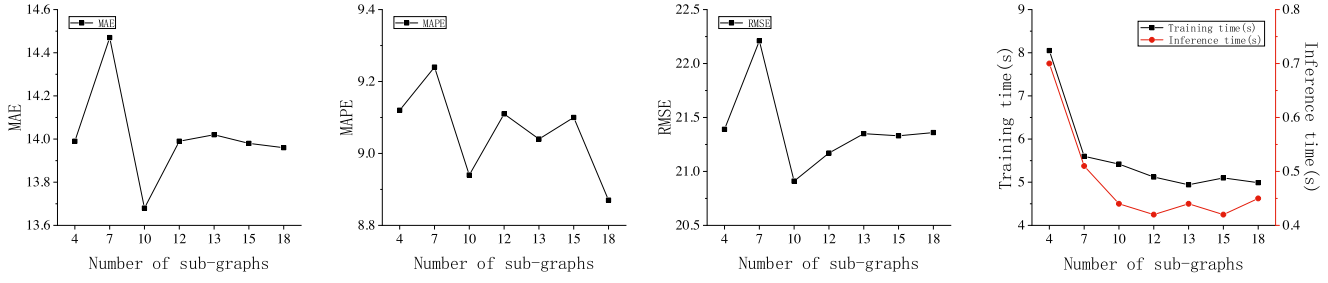


Fig. 7. Comparison of predicted performance over different number of sub-graphs [4, 7, 10, 12, 13, 15, 18] for PeMS08.

5.4.1. The number of connected spatial graphs

The number of spatial graphs determines the size of the asynchronous spatial–temporal graph. In our experiment, for the METR-LA dataset, we selected 2, 3, 4, 5, 6, 8, and 12 as the value of this parameter, respectively, and analyzed their influence on the model's performance. Fig. 4 compares the performances of the FedAGCN model on the METR-LA dataset under different numbers of spatial graphs. In the four sub-graphs, the horizontal axis represents the number of connected spatial graphs, and the vertical axis represents the value of each metric. From Fig. 4, for the METR-LA dataset, when the number of spatial graphs is larger, the prediction performance of the model is better. At the same time, the training time and inference time of the model decrease as the number of spatial graphs increases. To balance the prediction accuracy and time cost, on the METR-LA data set, we choose 8 as the value of this parameter.

Table 2
Comparison of prediction performance of different models.

Data	Models	MAE	RMSE	MAPE (%)	Training time (s)	Inference time (s)
METR-LA	T-GCN	3.39	5.29	8.39	9.5	2.01
	DCRNN	2.18	**3.78**	**5.19**	137.2	17.02
	Graph Wavenet	2.22	3.84	5.35	42.10	1.62
	GMAN	2.41	4.36	6.04	607.2	27.9
	FedAGCN	**2.17**	3.89	5.28	**5.02**	**0.45**
PEMS08	T-GCN	14.84	22.38	11.01	3.32	0.62
	DCRNN	12.95	20.03	8.40	70.0	8.06
	Graph Wavenet	**12.63**	**19.74**	**8.15**	17.87	0.67
	GMAN	14.77	22.25	11.23	251.1	11.3
	FedAGCN	13.75	20.89	8.85	**3.28**	**0.38**

The results on the PEMS08 dataset are shown in Fig. 5. Unlike the METR-LA dataset, the prediction performance of FedAGCN on PEMS08 fluctuates as the number of spatial graphs increases. We still choose to connect eight spatial graphs to form an asynchronous spatial–temporal graph.

5.4.2. The number of sub-graphs

FedAGCN divides the traffic road network into multiple sub-graphs, so the number of sub-graphs has a greater impact on the prediction performance. For the METR-LA dataset, we divide the traffic road network into 6, 9, 12, 15, 18, and 20 sub-graphs. Fig. 6 shows the prediction performance of FedAGCN on the METR-LA dataset. From Fig. 6(a), (b) and (c), when the number of sub-graphs is greater than or equal to 9, the prediction performance of the model is relatively stable. Fig. 6(d) shows that when the number of sub-graphs is less than 20, the time cost of the model decreases as the number of sub-graphs increases. In practical applications, each sub-graph needs an traffic node with certain computing capability as the representative node. Therefore, the number of sub-graphs is directly proportional to the cost of model deployment. Considering the balance of performance and cost, on the METR-LA dataset, we divide the traffic road network into 9 sub-graphs.

Fig. 7 shows the results of this experiment on the PEMS08 dataset. We divide the traffic road network into 4, 7, 10, 12, 13, 15, and 18 sub-graphs. For the same consideration, we set the number of sub-graphs of the FedAGCN model on this dataset to 10.

In addition, we apply grid search on optimal parameter selection and set the learning rate as 0.001, the batch size as 32, and the training epoch as 100.

5.5. Experimental results

Table 2 records one-step ahead (in a 5 min interval) forecasting results, training and inference time for FedAGCN and baseline methods. The bold part indicates that the model has the best performance. From Table 2 the prediction accuracy of FedAGCN is close to the state-of-the-art model. In addition, the training time and inference time of FedAGCN are much less than those of the existing deep learning models.

Among the models in Table 2, T-GCN, DCRNN, Graph Wavenet, and GMAN are all centralized deep learning models, while FedAGCN is a federated learning model. In addition to the temporal correlation of traffic data, these models also consider the spatial connection of the traffic road network. GMAN uses the attention mechanism to accurately model the impact of temporal and spatial factors in traffic data on traffic flow. Its prediction accuracy is greatly improved compared with the T-GCN model. The prediction accuracy of FedAGCN is very close to the latest deep learning model. For example, on the METR-LA dataset, the MAE of FedAGCN is 2.17, which is better than the GMAN model.

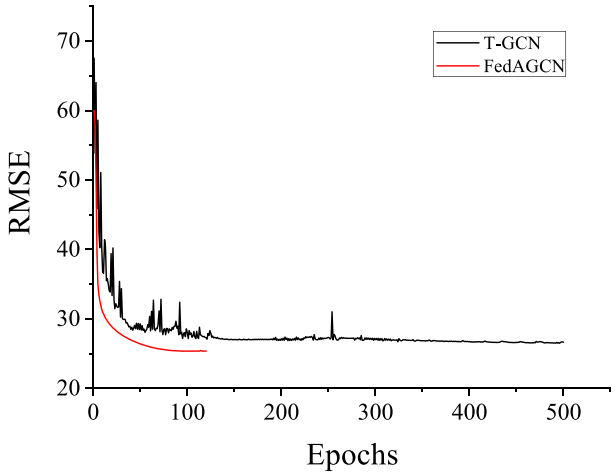


Fig. 8. Comparison of the training process of T-GCN and FedAGCN.

This proves that our model avoids the accuracy loss caused by using the federated learning strategy.

In addition, FedAGCN has apparent advantages in training time and inference time. In real-time traffic forecasting tasks, the time cost is a critical measure. Compared with FedAGCN, the inference time and deployment cost of other baseline models are relatively large. The iterative calculation method of DCRNN requires much time, resulting in a longer calculation time for the model. Graph Wavenet uses one-dimensional dilated causal convolution instead of RNN to model the time dependence of traffic data, which better reduces the calculation time of the model, but still cannot meet the needs of real-time prediction tasks. GMAN is not suitable for processing real-time prediction tasks because it needs to calculate the attention of each node in the spatial and temporal dimensions, which will take more computing time and space. It is worth noting that in Table 2, the training time of one epoch of T-GCN is very close to that of FedAGCN. However, the time required to train T-GCN is often several times that of FedAGCN. As shown in Fig. 8, FedAGCN has been fitted after 90 epochs, while T-GCN is fitted after about 400 epochs.

FedAGCN uses the sub-graph division strategy to divide the traffic network into multiple sub-graphs and then uses the strategy GraphFed to train the sub and cloud models. The model takes into account both the local spatial correlation and the global temporal correlation of traffic data. Dividing the overall traffic network into sub-graphs reduces the number of parameters of each sub-model, and at the same time, avoids the time overhead required for a large amount of data transmission. Therefore, the prediction time and deployment cost of FedAGCN are relatively

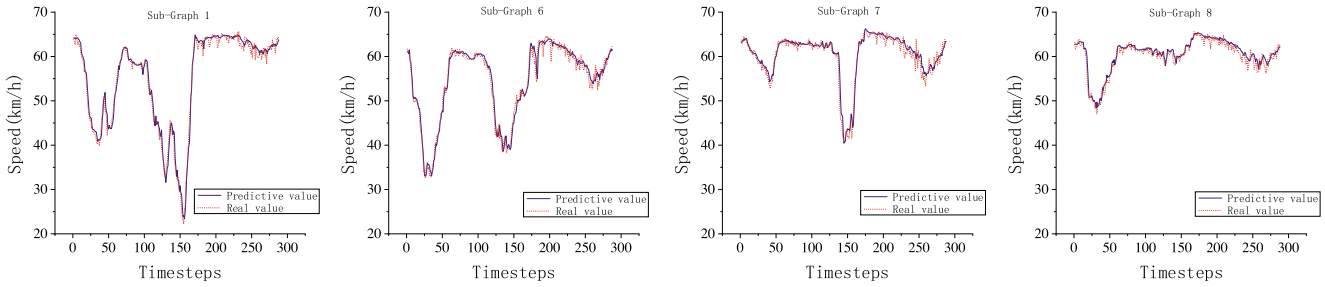
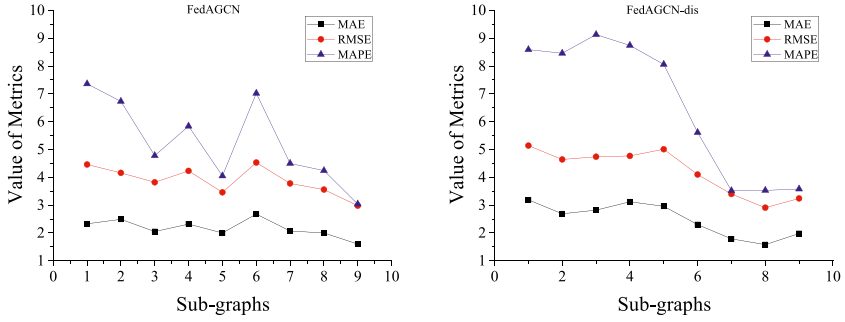


Fig. 9. Comparison of the predicted values and real values of 4 randomly selected sub-graphs in 288 continuous time steps.



(a) The Metrics of FedAGCN (b) The Metrics of FedAGCN-dis

Fig. 10. The prediction performance of FedAGCN and FedAGCN-dis on different sub-graphs.

Table 3
Ablation experiments.

Models	METR-LA		
	MAE	RMSE	MAPE (%)
FedAGCN-dis	2.53	4.23	6.50
FedAGCN-all	3.26	5.61	8.95
FedGRU	6.14	7.96	23.28
FedAGCN	**2.17**	**3.89**	**5.28**

low. Furthermore, FedAGCN considers the asynchronous spatial–temporal correlation in each sub-graph and directly models it, thus ensuring its prediction accuracy. It can be seen that from Table 2, FedAGCN requires less inference time and deployment costs than other baselines but maintains considerable accuracy.

We plot predicted values vs. real values of FedAGCN model on the METR-LA dataset in Fig. 9. Specifically, we select 4 sub-graphs (#1, 6, 7, 8) and arbitrary 288 continuous-time steps (1 day) to compare the real value with the predicted value. In the traffic network, the speed of different sub-graphs has different variation patterns. It can be seen from the figure that FedAGCN can adapt to the traffic data patterns of different sub-graphs and maintain high prediction accuracy. In Fig. 9(a), the traffic data fluctuate drastically, and the forecasting performance of FedAGCN drops slightly, but it still maintains a high accuracy rate.

5.6. Ablation experiments

The sub-graph division strategy and GraphFed strategy are the key components of FedAGCN. We designed ablation experiments to verify the effectiveness of the two key components. Table 3 shows the prediction performance of FedAGCN and three ablation models on the METR-LA datasets. It can be seen that compared with the ablation models, the prediction performance of FedAGCN has been greatly improved.

5.6.1. Effect analysis of sub-graph division strategy

To verify the effect of the sub-graph division strategy in FedAGCN, we replace it with other division strategies. Intuitively, there is a strong spatial correlation between the traffic sensor nodes close to each other. Therefore, an intuitive way to divide the sub-graph is to divide the traffic sensor nodes whose distance is less than a certain threshold into the same sub-graph. We call this strategy FedAGCN-dis. It can be seen from Table 3 that compared with the FedAGCN-dis model, the FedAGCN model achieves a lower prediction error. In Fig. 10, we plot the MAE and RMSE of each sub-graph of the two models on the METR-LA dataset. It can be seen that the prediction performance of each sub-graph of FedAGCN-dis fluctuates significantly, while FedAGCN is relatively stable. This shows that FedAGCN-dis's distance-based sub-graph division strategy is not workable. In some cases, adjacent nodes may sample conflicting signals, such as traffic detector nodes located on two lanes in different directions on the same road. FedAGCN uses a data-driven clustering algorithm to divide sub-graphs. The spatial correlation between traffic nodes can be fully explored.

The sub-graph division strategy divides the transportation network into multiple sub-graphs so that the model can efficiently utilize the spatial information of the transportation network. To verify the impact of modeling spatial correlation on model prediction performance, we compare FedAGCN with FedGRU [31] on the METR-LA dataset, and the experimental results are shown in Table 3. FedGRU creates a sub-model based on GRU on each traffic node, and the parameter update between sub-models also adopts a federated learning strategy. It can be seen from the table that the prediction performance of FedAGCN is significantly improved

compared to FedGRU, which verifies the necessity of modeling spatial dependencies.

5.6.2. Effect analysis of GraphFed strategy

To verify the effect of GraphFed, we use all the parameters in the FedAGCN model as global parameters to participate in the aggregation, which we call FedAGCN-all. It can be seen from Table 3 that the prediction accuracy of FedAGCN-all is much lower than that of the FedAGCN model, which shows that the participation of GCN related parameters in the global parameter update process has a negative impact on the prediction accuracy of the model. Because the spatial correlation varies within different clusters, and the aggregation of the relevant parameters of each sub-model will add interference, which reduces the model accuracy. The GraphFed strategy divides the model parameters into local and global parameters, which can help each representative node learns its spatial–temporal characteristics.

6. Conclusion

This paper proposes a new traffic flow prediction model FedAGCN based on federated learning and an improved Asynchronous Graph Convolutional Network. We first propose a sub-graph partitioning strategy for traffic networks to divide the traffic graph into multiple sub-graphs and then design a graph federated learning strategy GraphFed to solve the parameter update problem of GCN. Finally, we improve the ADGCN model and use it as a sub-model of the federated learning model. Experiments on two real traffic datasets show that FedAGCN achieves lower computational time overhead while maintaining higher prediction accuracy than existing centralized deep learning models.

FedAGCN is suitable for large-scale traffic prediction applications which are sensitive to time overhead. Meanwhile, FedAGCN uses new sampled data to fine-tune its parameters, so that it can continuously capture the varying spatial–temporal dependency of traffic data. Like many existing works, FedAGCN performs traffic predictions on one traffic parameter with a fixed graph structure. Thus it may lose its flexibility and robustness under the following challenging cases: (1) New features are available. (2) New data from another hub is available. (3) The original graph structure is changed after adding or reducing some traffic nodes. In the first case, we should retrain FedAGCN to learn different spatial–temporal dependencies on new traffic features. In the second case, we cannot directly deploy the trained model on new areas without optimizing strategies like parameter transfer learning due to the diverse graph structures between two hubs. Finally, in the third case, the changed graph structure will affect the extraction efficiency of the original model for spatial–temporal information. Thus we should reconstruct the graph structure and fine-tune the parameters of the pre-trained model. In future work, we will consider designing robust and scalable prediction models under the above challenging cases.

CRediT authorship contribution statement

Tao Qi: Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **Lingqiang Chen:** Methodology, Project administration, Writing – review & editing, Formal analysis. **Guanghui Li:** Supervision, Funding acquisition, Writing – review & editing. **Yijing Li:** Methodology, Resources. **Chenshu Wang:** Software, Validation, Writing – original draft.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work is funded in part by the National Natural Science Foundation of China (File no. 62072216).

References

[1] A. Zanella, N. Bui, A. Castellani, L. Vangelista, M. Zorzi, Internet of things for smart cities, IEEE Internet Things J. 1 (1) (2014) 22–32.
[2] W. Jiang, J. Luo, Graph neural network for traffic forecasting: A survey, Expert Syst. Appl. 207 (2022) 117921.
[3] M.N. Sadiku, M. Tembely, S.M. Musa, Internet of vehicles: An introduction, Int. J. Adv. Res. Comput. Sci. Softw. Eng. 8 (1) (2018) 11.
[4] R. Dai, S. Xu, Q. Gu, C. Ji, K. Liu, Hybrid spatio-temporal graph convolutional network: Improving traffic prediction with navigation data, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020, pp. 3074–3082.
[5] J. Zhu, X. Han, H. Deng, C. Tao, L. Zhao, P. Wang, T. Lin, H. Li, KST-GCN: A knowledge-driven spatial-temporal graph convolutional network for traffic forecasting, IEEE Trans. Intell. Transp. Syst. 23 (9) (2022) 15055–15065.
[6] L.R. Medsker, L. Jain, Recurrent neural networks, Des. Appl. 5 (2001) 64–67.
[7] F. Scarselli, M. Gori, A.C. Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model, IEEE Trans. Neural Netw. 20 (1) (2008) 61–80.
[8] Y. Li, Z. Hao, H. Lei, Survey of convolutional neural network, J. Comput. Appl. 36 (9) (2016) 2508–2515.
[9] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Artificial Intelligence and Statistics, PMLR, 2017, pp. 1273–1282.
[10] J. Liu, W. Guan, A summary of traffic flow forecasting methods, J. Highw. Transp. Res. Dev. 3 (2004) 82–85.
[11] M.M. Hamed, H.R. Al-Masaeid, Z.M.B. Said, Short-term prediction of traffic volume in urban arterials, J. Transp. Eng. 121 (3) (1995) 249–254.
[12] Y. Sun, B. Leng, W. Guan, A novel wavelet-SVM short-time passenger flow prediction in Beijing subway system, Neurocomputing 166 (2015) 109–121.
[13] Y. Lv, Y. Duan, W. Kang, Z. Li, F.-Y. Wang, Traffic flow prediction with big data: a deep learning approach, IEEE Trans. Intell. Transp. Syst. 16 (2) (2014) 865–873.
[14] M. Zhang, Z. Cui, M. Neumann, Y. Chen, An end-to-end deep learning architecture for graph classification, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 32, 2018.
[15] J. Johnson, A. Gupta, L. Fei-Fei, Image generation from scene graphs, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 1219–1228.
[16] B. Yu, H. Yin, Z. Zhu, Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting, 2017, arXiv preprint arXiv:1709.04875.
[17] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, H. Li, T-gcn: A temporal graph convolutional network for traffic prediction, IEEE Trans. Intell. Transp. Syst. 21 (9) (2019) 3848–3858.
[18] Y. Li, R. Yu, C. Shahabi, Y. Liu, Diffusion convolutional recurrent neural network: Data-driven traffic forecasting, in: International Conference on Learning Representations, 2018.
[19] Z. Wu, S. Pan, G. Long, J. Jiang, C. Zhang, Graph wavenet for deep spatial-temporal graph modeling, in: Proceedings of the 28th International Joint Conference on Artificial Intelligence, 2019, pp. 1907–1913.
[20] Z. Cui, R. Ke, Z. Pu, X. Ma, Y. Wang, Learning traffic as a graph: A gated graph wavelet recurrent neural network for network-scale traffic prediction, Transp. Res. C 115 (2020) 102620.
[21] Z. Pan, Y. Liang, W. Wang, Y. Yu, Y. Zheng, J. Zhang, Urban traffic prediction from spatio-temporal data using deep meta learning, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 1720–1730.
[22] C. Zheng, X. Fan, C. Wang, J. Qi, Gman: A graph multi-attention network for traffic prediction, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 34, 2020, pp. 1234–1241.
[23] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konečný, S. Mazzocchi, B. McMahan, et al., Towards federated learning at scale: System design, in: Proceedings of Machine Learning and Systems, Vol. 1, 2019, pp. 374–388.
[24] Q. Yang, Y. Liu, Y. Cheng, Y. Kang, T. Chen, H. Yu, Federated learning, Synth. Lect. Artif. Intell. Mach. Learn. 13 (3) (2019) 1–207.

[25] M. Ammad-Ud-Din, E. Ivannikova, S.A. Khan, W. Oyomno, Q. Fu, K.E. Tan, A. Flanagan, Federated collaborative filtering for privacy-preserving personalized recommendation system, 2019, arXiv preprint arXiv:1901.09888.

[26] A. Hard, K. Rao, R. Mathews, S. Ramaswamy, F. Beaufays, S. Augenstein, H. Eichner, C. Kiddon, D. Ramage, Federated learning for mobile keyboard prediction, 2018, arXiv preprint arXiv:1811.03604.

[27] S. Ramaswamy, R. Mathews, K. Rao, F. Beaufays, Federated learning for emoji prediction in a mobile keyboard, 2019, arXiv preprint arXiv:1906.04329.

[28] L. Huang, A.L. Shea, H. Qian, A. Masurkar, H. Deng, D. Liu, Patient clustering improves efficiency of federated machine learning to predict mortality and hospital stay time using distributed electronic medical records, J. Biomed. Inform. 99 (2019) 103291.

[29] S. Silva, B.A. Gutman, E. Romero, P.M. Thompson, A. Altmann, M. Lorenzi, Federated learning in distributed medical databases: Meta-analysis of large-scale subcortical brain data, in: 2019 IEEE 16th International Symposium on Biomedical Imaging (ISBI 2019), IEEE, 2019, pp. 270–274.

[30] Y. Zhao, J. Zhao, L. Jiang, R. Tan, D. Niyato, Mobile edge computing, blockchain and reputation-based crowdsourcing iot federated learning: A secure, decentralized and privacy-preserving system, 2020.

[31] Y. Liu, J. James, J. Kang, D. Niyato, S. Zhang, Privacy-preserving traffic flow prediction: A federated learning approach, IEEE Internet Things J. 7 (8) (2020) 7751–7763.

[32] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, S.Y. Philip, A comprehensive survey on graph neural networks, IEEE Trans. Neural Netw. Learn. Syst. 32 (1) (2020) 4–24.

[33] G. Karypis, V. Kumar, A fast and high quality multilevel scheme for partitioning irregular graphs, SIAM J. Sci. Comput. 20 (1) (1998) 359–392.

[34] I.S. Dhillon, Y. Guan, B. Kulis, Weighted graph cuts without eigenvectors a multilevel approach, IEEE Trans. Pattern Anal. Mach. Intell. 29 (11) (2007) 1944–1957.

[35] T. Qi, G. Li, L. Chen, Y. Xue, ADGCN: An asynchronous dilation graph convolutional network for traffic flow prediction, IEEE Internet Things J. 9 (5) (2021) 4001–4014.

[36] C. Chen, K. Petty, A. Skabardonis, P. Varaiya, Z. Jia, Freeway performance measurement system: mining loop detector data, Transp. Res. Rec. 1748 (1) (2001) 96–102.

[37] D.I. Shuman, S.K. Narang, P. Frossard, A. Ortega, P. Vandergheynst, The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains, IEEE Signal Process. Mag. 30 (3) (2013) 83–98.